

Friedman Test (with Post-Hoc Tests)

v1.0.0

Allan Omondi

21 June 2026

Table of contents

1	What Is the Friedman Test?	3
1.1	Why Not Simply Reuse the One-Way ANOVA?	3
1.2	A Note on Terminology: Blocks, Treatments, and Subjects	4
2	Prepare the Dataset for the Friedman Test	4
3	Initial EDA	7
3.1	<u>Measures of Frequency</u>	8
3.2	<u>Measures of Central Tendency</u>	9
3.3	<u>Measures of Distribution</u>	10
3.3.1	Variance and Standard Deviation	10
3.3.2	Kurtosis and Skewness (Pearson, “type 2”)	11
3.4	<u>Basic Visualizations</u>	12
3.4.1	Histogram (by Algorithm)	12
3.4.2	Box and Whisker Plot, with Raw Points (by Algorithm)	13
3.4.3	Missing Data Plot	15
3.4.4	Profile Plot (the Key Visualization for a Block Design)	17
4	Statistical Test: Friedman Test	19
4.1	The Friedman Test	19
4.2	Average Ranks	20
4.3	Effect Size: Kendall’s W	21
4.4	Post-Hoc Test: Nemenyi Test	21
4.5	Alternative Post-Hoc Test: Pairwise Wilcoxon Signed-Rank Tests	23
5	Diagnostic EDA (Model Diagnostic)	24
5.1	<u>Test of Linearity — Not Applicable</u>	24
5.2	<u>Test of Normality — Not Required</u>	24

5.3	Test of Homoscedasticity — Not Required	24
5.4	Completeness of the Block Design	25
5.5	Check for Tied Ranks Within Blocks	25
5.6	Test of Non-Additivity (Block $\times$ Treatment Interaction)	26
6	Interpretation of the Results	26
6.1	Friedman Test and Post-Hoc Comparisons	28
6.1.1	Technical Academic Report (Based on APA Guidelines of Reporting Results)	28
6.1.2	Non-Technical Business Report	29
6.1.3	Recommendations	30
7	References and Further Reading	30

```
# `installed.packages()` retrieves a matrix of all installed packages
# `[ , "Package"]` extracts on the "Package" column from the matrix of all
# packages. This contains the name of the package.
# The %in% operator is used to test if the specified package is in the matrix of
# all packages
# `dependencies = TRUE` instructs R to install not only the specified package but
# also its dependencies

if (!"pacman" %in% installed.packages()[ , "Package"]) {
  install.packages("pacman", dependencies = TRUE)
  library("pacman")
}

# For reading data from a CSV file
pacman::p_load("readr")
# For data wrangling: aggregating runs into blocks and reshaping long <-> wide
pacman::p_load("dplyr")
pacman::p_load("tidyr")
# For data visualization
pacman::p_load("ggplot2")
# For data visualization of missing data
pacman::p_load("naniar")
# For skewness and kurtosis
pacman::p_load("e1071")

# Note on package choice: unlike the ANOVA notebook, this notebook does not
# load "car" (Levene's test) or normality-testing workflows as required
# diagnostics, because the Friedman test does not assume homogeneity of
# variance or normally distributed residuals. Where those concepts are
```

```
# revisited below, it is for contrast and teaching purposes only, not because
# the Friedman test requires them. The post-hoc test used below (Nemenyi) is
# computed directly from R's built-in studentized range distribution
# functions (`qtukey()`, `ptukey()`), so no additional post-hoc package is
# required either.
```

1 What Is the Friedman Test?

Purpose: Tests whether three or more *related* (matched, repeated, or blocked) sets of measurements have the same distribution, using ranks rather than raw values. It is the non-parametric counterpart to a one-way repeated-measures ANOVA, and is also describable as the non-parametric counterpart to a two-way ANOVA without replication, where one factor (the block) is not of substantive interest in itself.

Variables: One categorical “treatment” factor (three or more related levels); one “block” factor identifying which observations are matched to one another; one outcome variable measured once per block-by-treatment combination.

Output: A chi-squared test statistic with $k - 1$ degrees of freedom (k = number of treatments) and an associated p-value; if significant, followed by a post-hoc test (e.g., the Nemenyi test) to identify which specific treatment pairs differ.

Example: Comparing classification Accuracy across four algorithms, where the same ten datasets are used to evaluate every algorithm — so each dataset acts as its own “block,” and the four algorithm scores within a block are matched rather than independent.

1.1 Why Not Simply Reuse the One-Way ANOVA?

The ANOVA notebook’s one-way ANOVA tested `Accuracy ~ Algorithm` by treating all 200 rows as independent observations. Re-examine that design with the block structure in mind: every one of the ten datasets appears under every one of the four algorithms. The same dataset is not an independent piece of evidence each time it is reused — it is the same “subject” measured four times, once per algorithm. This is precisely the situation a one-way independent-groups ANOVA is not designed for, and precisely the situation the Friedman test (or a repeated-measures ANOVA) is designed for.

Two further reasons commonly motivate choosing the Friedman test over a repeated-measures ANOVA specifically:

1. **The outcome is ordinal, or its distribution is markedly non-normal**, and a rank-based test is preferred to a test that assumes interval-level, approximately normal data.

2. **The sample is small** (as here: only ten datasets), where the normality assumption of a parametric repeated-measures ANOVA is hard to verify with confidence, and a distribution-free test is the safer default.

Research question used in this notebook: *Does classification accuracy differ across the four algorithms when each algorithm is evaluated on the same ten datasets?*

1.2 A Note on Terminology: Blocks, Treatments, and Subjects

Different fields use different words for the same three roles in this design:

Role	This notebook	Common synonym
The matched unit	Block	Subject, case, participant
The thing being compared	Treatment	Condition, group, algorithm
One matched score	Cell	Observation, measurement

Here, **Dataset** plays the role of the block (it is the “subject” that is repeatedly measured), and **Algorithm** plays the role of the treatment.

2 Prepare the Dataset for the Friedman Test

The Friedman test requires a **complete block design**: exactly one numeric score per block-by-treatment combination — no fewer, no more. `ml_results.csv` contains five replicate runs per Dataset–Algorithm cell, which is too many; the Friedman test has no mechanism for handling repeated values within a single block-treatment cell, because it works by ranking the treatments *within each block*, and a block can only contain one rank-eligible value per treatment.

The five runs are not discarded; they are summarized into one representative value per cell using the mean, exactly mirroring how the ANOVA notebook already defined “run” as a replicate rather than a new experimental unit.

```
ml_performance_data <- read_csv(
  "data/ml_results.csv",
  col_types = cols(
    Dataset = col_factor(
      levels = c("Dataset_1", "Dataset_2", "Dataset_3",
                 "Dataset_4", "Dataset_5", "Dataset_6",
                 "Dataset_7", "Dataset_8", "Dataset_9",
                 "Dataset_10")),
```

```

Run      = col_factor(levels = c("1", "2", "3", "4", "5")),
Algorithm = col_factor(levels = c("Logistic Regression",
                                   "Random Forest",
                                   "SVM", "Gradient Boosting")),

Accuracy = col_double(),
Precision = col_double(),
Recall    = col_double()
)
)

head(ml_performance_data)

```

```

#> # A tibble: 6 x 6
#>   Dataset Run   Algorithm Accuracy Precision Recall
#>   <fct>   <fct> <fct>         <dbl>     <dbl>   <dbl>
#> 1 Dataset_1 1     Logistic Regression 0.703      0.747 0.702
#> 2 Dataset_2 1     Logistic Regression 0.713      0.739 0.690
#> 3 Dataset_3 1     Logistic Regression 0.767      0.761 0.754
#> 4 Dataset_4 1     Logistic Regression 0.722      0.733 0.722
#> 5 Dataset_5 1     Logistic Regression 0.724      0.716 0.737
#> 6 Dataset_6 1     Logistic Regression 0.771      0.762 0.738

```

```

# `group_by()` followed by `summarise()` collapses the 5 runs per
# Dataset-Algorithm cell into a single mean Accuracy value. This produces
# exactly one score per block (Dataset) per treatment (Algorithm): the
# complete block design the Friedman test requires.
ml_blocked_data <- ml_performance_data |>
  dplyr::group_by(Dataset, Algorithm) |>
  dplyr::summarise(Accuracy = mean(Accuracy), .groups = "drop")

dim(ml_blocked_data)

```

```

#> [1] 40 3

```

```

head(ml_blocked_data, 8)

```

```

#> # A tibble: 8 x 3
#>   Dataset Algorithm Accuracy
#>   <fct>   <fct>         <dbl>
#> 1 Dataset_1 Logistic Regression 0.716

```

```
#> 2 Dataset_1 Random Forest      0.787
#> 3 Dataset_1 SVM                0.756
#> 4 Dataset_1 Gradient Boosting  0.815
#> 5 Dataset_2 Logistic Regression 0.716
#> 6 Dataset_2 Random Forest      0.769
#> 7 Dataset_2 SVM                0.751
#> 8 Dataset_2 Gradient Boosting  0.814
```

The result is 40 rows: 10 datasets $\times$ 4 algorithms, one mean accuracy value per combination — the long-format equivalent of the block design.

```
# `pivot_wider()` reshapes the long block-level data into a Dataset x
# Algorithm matrix: one row per block, one column per treatment. This wide
# layout is both easier to inspect by eye and the layout `friedman.test()`
# expects when supplied as a matrix.
ml_blocked_wide <- ml_blocked_data |>
  tidyr::pivot_wider(names_from = Algorithm, values_from = Accuracy)

ml_blocked_wide
```

```
#> # A tibble: 10 x 5
#>   Dataset   `Logistic Regression` `Random Forest`   SVM `Gradient Boosting`
#>   <fct>                <dbl>          <dbl> <dbl>          <dbl>
#> 1 Dataset_1           0.716            0.787 0.756          0.815
#> 2 Dataset_2           0.716            0.769 0.751          0.814
#> 3 Dataset_3           0.723            0.783 0.727          0.803
#> 4 Dataset_4           0.735            0.778 0.734          0.824
#> 5 Dataset_5           0.726            0.775 0.737          0.793
#> 6 Dataset_6           0.728            0.795 0.751          0.814
#> 7 Dataset_7           0.732            0.791 0.730          0.828
#> 8 Dataset_8           0.698            0.788 0.745          0.800
#> 9 Dataset_9           0.716            0.784 0.761          0.822
#> 10 Dataset_10        0.719            0.793 0.733          0.797
```

```
# friedman.test() accepts a matrix where rows = blocks and columns =
# treatments. The Dataset identifiers are moved to the row names so that
# only the four numeric algorithm columns remain in the matrix itself.
ml_matrix <- as.matrix(ml_blocked_wide[, -1])
rownames(ml_matrix) <- ml_blocked_wide$Dataset

ml_matrix
```

#>	Logistic Regression	Random Forest	SVM	Gradient Boosting
#> Dataset_1	0.7159652	0.7868472	0.7558694	0.8146571
#> Dataset_2	0.7164520	0.7685618	0.7506401	0.8143419
#> Dataset_3	0.7233792	0.7829864	0.7266837	0.8026975
#> Dataset_4	0.7349963	0.7779438	0.7344218	0.8241552
#> Dataset_5	0.7258677	0.7749276	0.7365412	0.7930516
#> Dataset_6	0.7282849	0.7954626	0.7512276	0.8143312
#> Dataset_7	0.7316855	0.7913931	0.7298026	0.8276879
#> Dataset_8	0.6983588	0.7883082	0.7451831	0.7997264
#> Dataset_9	0.7161022	0.7839948	0.7609046	0.8224395
#> Dataset_10	0.7192291	0.7934971	0.7325558	0.7971075

3 Initial EDA

The Initial EDA in the ANOVA notebook profiled the raw 200-row dataset. That EDA is not repeated here in full, since it was already produced once and the conclusions about the raw data's data types, frequencies, and basic visual shape do not change. What changes is the unit of analysis: the Friedman test is conducted on the 40-row block-level dataset (`ml_blocked_data`), so the EDA below profiles *that* dataset, since it is the one that determines whether the data are in a form suitable for the test.

View the Dimensions

```
dim(ml_blocked_data)
```

```
#> [1] 40 3
```

View the Data Types

```
sapply(ml_blocked_data, class)
```

```
#> Dataset Algorithm Accuracy
#> "factor" "factor" "numeric"
```

```
str(ml_blocked_data)
```

```
#> tibble [40 x 3] (S3: tbl_df/tbl/data.frame)
#> $ Dataset : Factor w/ 10 levels "Dataset_1","Dataset_2",...: 1 1 1 1 2 2 2 2 3 3 ...
#> $ Algorithm: Factor w/ 4 levels "Logistic Regression",...: 1 2 3 4 1 2 3 4 1 2 ...
#> $ Accuracy : num [1:40] 0.716 0.787 0.756 0.815 0.716 ...
```

Descriptive Statistics

Understanding your data can lead to:

- **Data cleaning:** To remove extreme outliers or impute missing data.
- **Data transformation:** To reduce skewness
- **Hypothesis formulation:** Formulate a hypothesis based on the patterns you identify
- **Choosing the appropriate statistical test:** You may notice properties of the data such as distributions or data types that suggest the use of parametric or non-parametric statistical tests and algorithms

Descriptive statistics can be used to understand your data. Typical descriptive statistics include:

1. **Measures of frequency:** count and percent
2. **Measures of central tendency:** mean, median, and mode
3. **Measures of distribution/dispersion/spread/scatter/variability:** minimum, quartiles, maximum, variance, standard deviation, coefficient of variation, range, interquartile range (IQR) [includes a box and whisker plot for visualization], kurtosis, skewness [includes a histogram for visualization].
4. **Measures of relationship:** covariance and correlation [includes a correlation plot and a scatter plot for visualization].

3.1 Measures of Frequency

Since the block design is, by construction, perfectly balanced (one observation per Dataset-Algorithm cell), the frequency table below exists mainly as a confirmation step: every algorithm should show exactly 10 observations (once per dataset), and every dataset should show exactly 4 observations (once per algorithm). A mismatch here would indicate an incomplete block design, which would invalidate the standard Friedman test.

```
cbind(  
  frequency = table(ml_blocked_data$Algorithm),  
  percentage = prop.table(table(ml_blocked_data$Algorithm)) *100  
)
```

```
#>               frequency percentage  
#> Logistic Regression      10         25  
#> Random Forest           10         25  
#> SVM                     10         25  
#> Gradient Boosting       10         25
```


3.2 Measures of Central Tendency

```
summary(ml_blocked_data)
```

```
#>      Dataset      Algorithm      Accuracy
#> Dataset_1: 4  Logistic Regression:10  Min.   :0.6984
#> Dataset_2: 4  Random Forest      :10  1st Qu.:0.7312
#> Dataset_3: 4  SVM                :10  Median  :0.7647
#> Dataset_4: 4  Gradient Boosting  :10  Mean    :0.7647
#> Dataset_5: 4                                     3rd Qu.:0.7940
#> Dataset_6: 4                                     Max.    :0.8277
#> (Other)      :16
```

```
ml_blocked_data |>
  dplyr::group_by(Algorithm) |>
  dplyr::summarise(
    Mean    = mean(Accuracy),
    Median  = median(Accuracy),
    SD      = sd(Accuracy),
    Min     = min(Accuracy),
    Max     = max(Accuracy)
  )
```

```
#> # A tibble: 4 x 6
#>   Algorithm      Mean Median      SD   Min   Max
#>   <fct>          <dbl> <dbl>   <dbl> <dbl> <dbl>
#> 1 Logistic Regression 0.721  0.721 0.0104  0.698 0.735
#> 2 Random Forest      0.784  0.785 0.00856 0.769 0.795
#> 3 SVM                0.742  0.741 0.0119  0.727 0.761
#> 4 Gradient Boosting  0.811  0.814 0.0122  0.793 0.828
```

The first 5 observations (rows) in the dataset:

```
head(ml_blocked_data, 5)
```

```
#> # A tibble: 5 x 3
#>   Dataset      Algorithm      Accuracy
#>   <fct>      <fct>          <dbl>
#> 1 Dataset_1 Logistic Regression  0.716
```

```
#> 2 Dataset_1 Random Forest      0.787
#> 3 Dataset_1 SVM                0.756
#> 4 Dataset_1 Gradient Boosting  0.815
#> 5 Dataset_2 Logistic Regression 0.716
```

The last 5 observations (rows) in the dataset:

```
tail(ml_blocked_data, 5)
```

```
#> # A tibble: 5 x 3
#>   Dataset      Algorithm      Accuracy
#>   <fct>       <fct>         <dbl>
#> 1 Dataset_9   Gradient Boosting    0.822
#> 2 Dataset_10 Logistic Regression  0.719
#> 3 Dataset_10 Random Forest    0.793
#> 4 Dataset_10 SVM              0.733
#> 5 Dataset_10 Gradient Boosting 0.797
```

3.3 Measures of Distribution

Measuring the variability in the dataset is important because the amount of variability determines **how well you can generalize** results from the sample to a new observation in the population.

Low variability is ideal because it means that you can better predict information about the population based on the sample data. High variability means that the values are less consistent, thus making it harder to make predictions.

The syntax `dataset[rows, columns]` can be used to specify the exact rows and columns to be considered. `dataset[, columns]` implies all rows will be considered. For example, specifying `BostonHousing[, -4]` implies all the columns except column number 4. This can also be stated as `BostonHousing[, c(1,2,3,5,6,7,8,9,10,11,12,13,14)]`. This allows us to perform calculations on only columns that are numeric, thus leaving out the columns termed as “factors” (categorical) or those that have a string data type.

These measures are computed per algorithm (10 values each: one per dataset), since the variability that matters for the Friedman test is the spread of scores *within* each treatment across blocks, not the spread of the pooled column.

3.3.1 Variance and Standard Deviation

```
ml_blocked_data |>
  dplyr::group_by(Algorithm) |>
  dplyr::summarise(Variance = var(Accuracy), SD = sd(Accuracy))
```

```
#> # A tibble: 4 x 3
#>   Algorithm      Variance      SD
#>   <fct>          <dbl>    <dbl>
#> 1 Logistic Regression 0.000109 0.0104
#> 2 Random Forest      0.0000733 0.00856
#> 3 SVM                0.000142 0.0119
#> 4 Gradient Boosting  0.000148 0.0122
```

3.3.2 Kurtosis and Skewness (Pearson, “type 2”)

The Kurtosis informs us of how often outliers occur in the results. There are different formulas for calculating kurtosis. Specifying “type = 2” allows us to use the 2nd formula which is the same kurtosis formula used in other statistical software like SPSS and SAS. It is referred to as “Pearson’s definition of kurtosis”.

In “type = 2” (used in SPSS and SAS):

1. Kurtosis < 3 implies a low number of outliers → platykurtic
2. Kurtosis = 3 implies a medium number of outliers → mesokurtic
3. Kurtosis > 3 implies a high number of outliers → leptokurtic

High kurtosis (leptokurtic) affects models that are sensitive to outliers. Estimates of the variance are also inflated. Low kurtosis (platykurtic) implies a possible underestimation of real-world variability. The typical remedy includes trimming outliers or using robust statistical methods that are less affected by outliers.

With only 10 values per algorithm, kurtosis and skewness estimates are imprecise and should be read as a rough indication of shape rather than a precise diagnostic. They are included here for completeness and continuity with the other notebooks, not because the Friedman test depends on them.

```
ml_blocked_data |>
  dplyr::group_by(Algorithm) |>
  dplyr::summarise(
    Skewness = e1071::skewness(Accuracy, type = 2),
    Kurtosis = e1071::kurtosis(Accuracy, type = 2)
  )
```

```
#> # A tibble: 4 x 3
#>   Algorithm      Skewness Kurtosis
#>   <fct>          <dbl>    <dbl>
#> 1 Logistic Regression -0.928    1.58
#> 2 Random Forest      -0.566   -0.394
#> 3 SVM                 0.211   -1.50
#> 4 Gradient Boosting  -0.155   -1.47
```

3.4 Basic Visualizations

3.4.1 Histogram (by Algorithm)

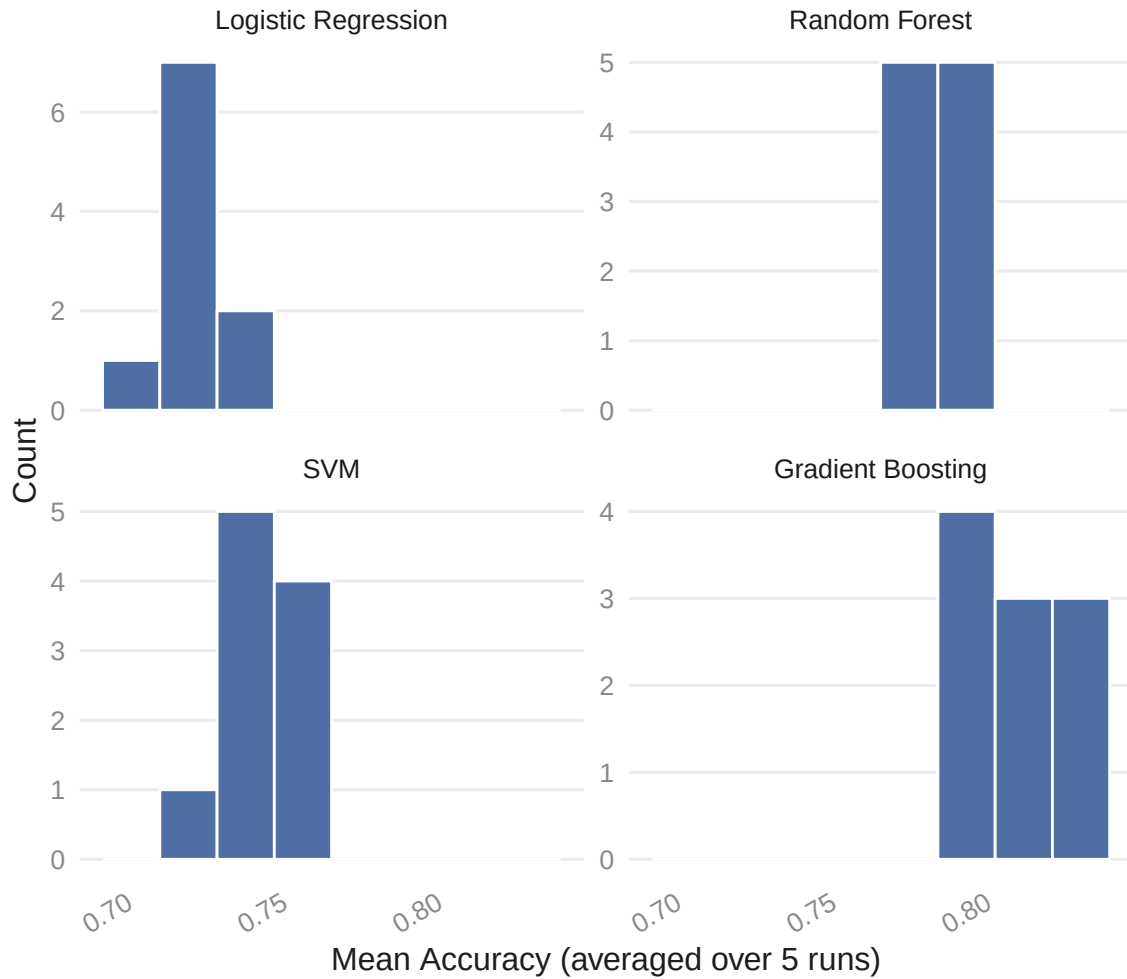
```
ggplot2::ggplot(ml_blocked_data, aes(x = Accuracy)) +
  geom_histogram(bins = 8, fill = "#4F6EA4", color = "#FFFFFF") +
  facet_wrap(~ Algorithm, scales = "free_y") +
  labs(
    title = "Distribution of Block-Level Mean Accuracy by Algorithm",
    subtitle = "EDA for shape and outliers, n = 10 datasets per algorithm",
    x = "Mean Accuracy (averaged over 5 runs)",
    y = "Count",
    caption = paste0("Source: ML Model Performance Data (block-level) | n = ",
                     nrow(ml_blocked_data))
  ) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title       = element_text(face = "bold", color = "#1D1D1D"),
    axis.title       = element_text(color = "#1D1D1D"),
    axis.text        = element_text(color = "#898989"),
    axis.text.x      = element_text(angle = 30, hjust = 1),
    plot.caption     = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x = element_blank(),
    panel.grid.minor.x = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    panel.grid.minor.y = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    # axis.line.x      = element_line(color = "#1D1D1D"),
    # axis.line.y      = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
```

```
#                               linewidth = 0.5)
)
```

Distribution of Block-Level Mean Accuracy by Algorithm

EDA for shape and outliers, n = 10 datasets per algorithm



Source: ML Model Performance Data (block-level) | n = 40

3.4.2 Box and Whisker Plot, with Raw Points (by Algorithm)

```
ggplot2::ggplot(ml_blocked_data, aes(x = Algorithm, y = Accuracy)) +
  stat_boxplot(geom = "errorbar", width = 0.15, color = "#1D1D1D") +
```

```

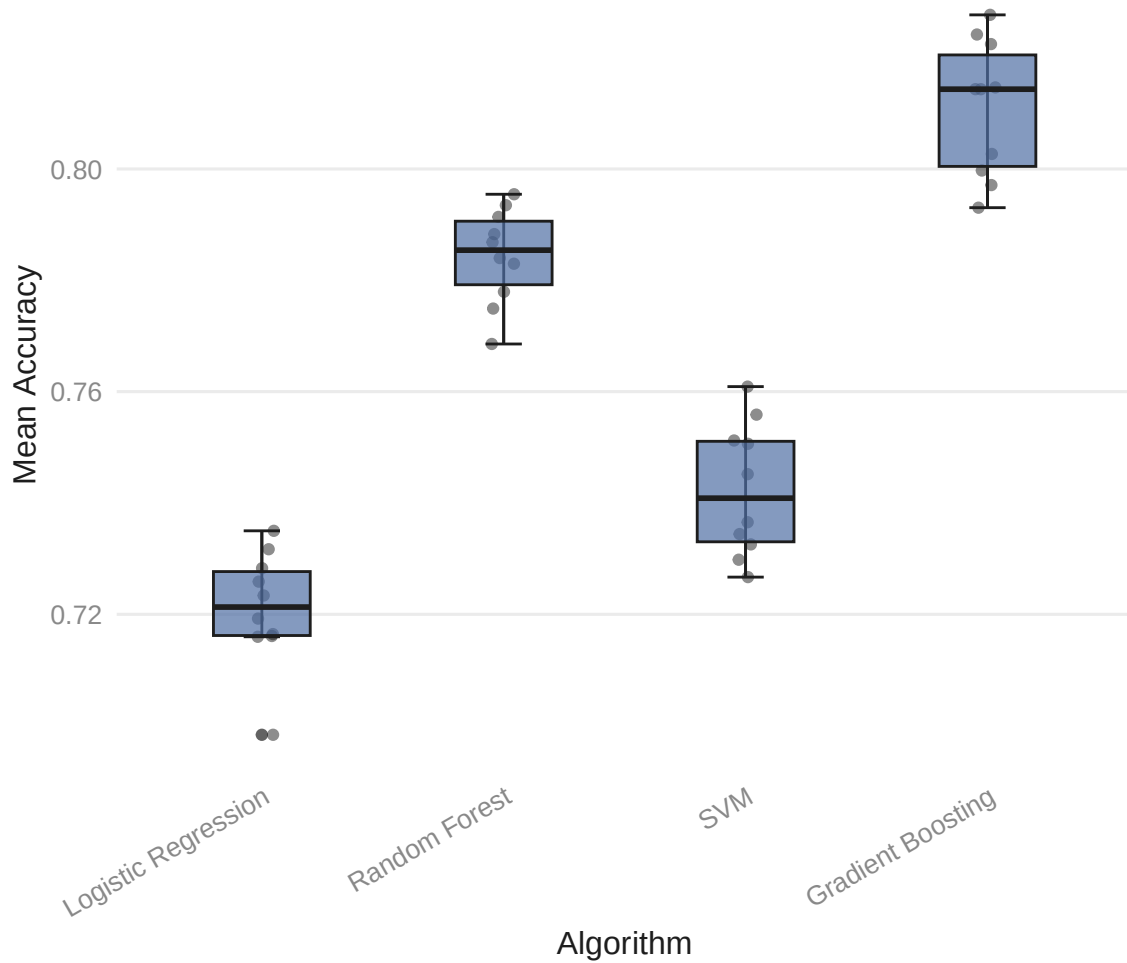
geom_jitter(width = 0.05, alpha = 0.5, color = "#1D1D1D", size = 1.5) +
geom_boxplot(fill = "#4F6EA4", color = "#1D1D1D", width = 0.4, alpha = 0.7) +
labs(
  title = "Block-Level Mean Accuracy by Algorithm",
  subtitle = "Each point is one dataset's mean accuracy (averaged over 5 runs)",
  x = "Algorithm",
  y = "Mean Accuracy",
  caption = paste0("Source: ML Model Performance Data (block-level) | n = ",
                    nrow(ml_blocked_data))
) +
theme_minimal(base_size = 12) +

theme(
  plot.title          = element_text(face = "bold", color = "#1D1D1D"),
  axis.title          = element_text(color = "#1D1D1D"),
  axis.text           = element_text(color = "#898989"),
  axis.text.x         = element_text(angle = 30, hjust = 1),
  plot.caption        = element_text(color = "#898989"),
  # removes vertical gridlines
  panel.grid.major.x  = element_blank(),
  panel.grid.minor.x  = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge bar heights against the y-axis scale
  panel.grid.minor.y  = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  # axis.line.x        = element_line(color = "#1D1D1D"),
  # axis.line.y        = element_line(color = "#1D1D1D")
  # panel.border = element_rect(color = "#1D1D1D", fill = NA,
  #                               linewidth = 0.5)
)

```

Block-Level Mean Accuracy by Algorithm

Each point is one dataset's mean accuracy (averaged over 5 runs)



Source: ML Model Performance Data (block-level) | n = 40

3.4.3 Missing Data Plot

```
naniar::vis_miss(ml_blocked_wide) +  
  labs(  
    title = "Missing Data Overview (Block Design)",  
    subtitle = "EDA for missingness before the Friedman test;\nany missing cell breaks the c",  
    caption = paste0("Source: ML Model Performance Data (block-level) | n = ",  
                      nrow(ml_blocked_wide))
```

```

) +
scale_fill_manual(values = c("Present" = "#4F6EA4", "Missing" = "#E03C31")) +
theme_minimal(base_size = 12) +

theme(
  plot.title          = element_text(face = "bold", color = "#1D1D1D"),
  axis.title          = element_text(color = "#1D1D1D"),
  axis.text           = element_text(color = "#898989"),
  axis.text.x         = element_text(angle = 90, hjust = 0),
  plot.caption        = element_text(color = "#898989"),
  # removes vertical gridlines
  panel.grid.major.x  = element_blank(),
  panel.grid.minor.x  = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge bar heights against the y-axis scale
  panel.grid.minor.y  = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  # axis.line.x        = element_line(color = "#1D1D1D"),
  # axis.line.y        = element_line(color = "#1D1D1D")
  # panel.border = element_rect(color = "#1D1D1D", fill = NA,
  #                               linewidth = 0.5)
)

```


Missing Data Overview (Block Design)

EDA for missingness before the Friedman test;
any missing cell breaks the complete block design



Source: ML Model Performance Data (block-level) | n = 10

3.4.4 Profile Plot (the Key Visualization for a Block Design)

A profile plot connects, for each block, its score across all treatments. It serves a different purpose to the box plot above: rather than comparing the overall spread of each algorithm, it shows whether the relative ordering of algorithms is *consistent from dataset to dataset*. Lines that run roughly parallel and do not cross indicate a stable ranking across blocks — the pattern the Friedman test is well-suited to detect, and good visual evidence that aggregating into ranks will not discard meaningful information.

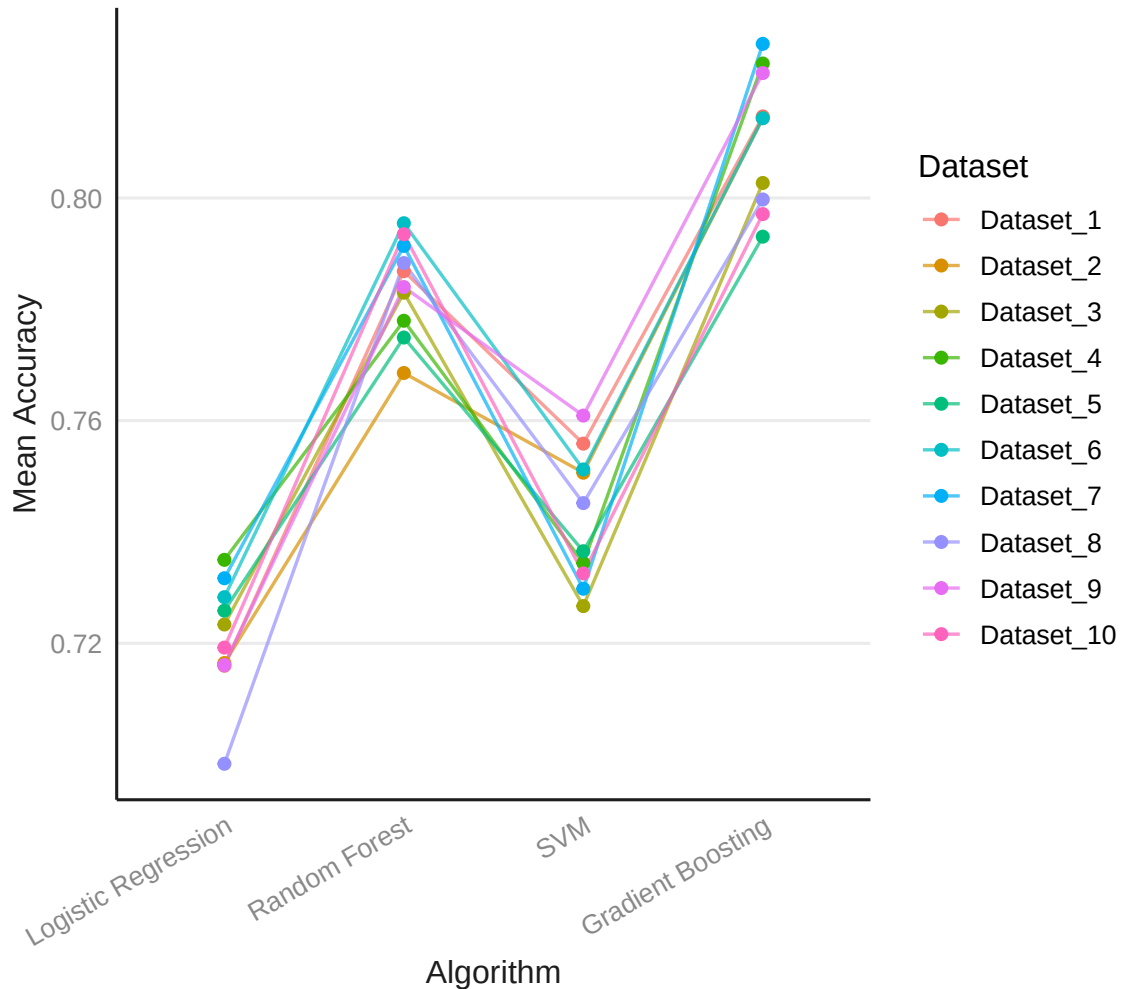
```

ggplot2::ggplot(ml_blocked_data,
                aes(x = Algorithm, y = Accuracy, group = Dataset, color = Dataset)) +
  geom_line(alpha = 0.7, linewidth = 0.6) +
  geom_point(size = 1.8) +
  labs(
    title = "Profile Plot: Mean Accuracy by Algorithm, One Line per Dataset",
    subtitle = "EDA for consistency of ranking across blocks before the Friedman test",
    x = "Algorithm",
    y = "Mean Accuracy",
    caption = paste0("Source: ML Model Performance Data (block-level) | n = ",
                     nrow(ml_blocked_data), " (10 datasets x 4 algorithms)")
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption        = element_text(color = "#898989"),
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    panel.grid.minor.y  = element_blank(),
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D"),
    legend.position     = "right"
  )

```

Profile Plot: Mean Accuracy by Algorithm, One Line per Dataset

EDA for consistency of ranking across blocks before the Friedman test



Model Performance Data (block-level) | n = 40 (10 datasets x 4 algorithms)

4 Statistical Test: Friedman Test

4.1 The Friedman Test

Purpose: Tests whether the central tendency of three or more related sets of scores is the same, by ranking the treatments within each block and testing whether the average ranks differ more than would be expected by chance.

Variables: One treatment factor (Algorithm, 4 levels); one block factor (Dataset, 10 levels);

one continuous outcome (mean Accuracy), exactly one value per block-treatment cell.

Output: A chi-squared statistic with $k - 1$ degrees of freedom and a p-value; if significant, followed by a post-hoc test to identify which specific algorithm pairs differ.

Example: Testing whether the four algorithms' accuracy ranks differ consistently across the ten datasets.

```
friedman_result <- friedman.test(ml_matrix)
friedman_result
```

```
#>
#> Friedman rank sum test
#>
#> data:  ml_matrix
#> Friedman chi-squared = 28.08, df = 3, p-value = 3.494e-06
```

`friedman.test()` reports three quantities: the chi-squared statistic, the degrees of freedom ($k - 1$, where k is the number of treatments), and the p-value. Internally, the test ranks the four algorithm scores within each dataset (1 = lowest accuracy, 4 = highest), sums the ranks for each algorithm across all ten datasets, and tests whether those rank sums are more unequal than chance would produce if the algorithm rankings within each dataset were randomly shuffled.

4.2 Average Ranks

The Friedman test statistic is built from each algorithm's average rank across blocks; reporting these alongside the test statistic is standard practice (especially in the machine learning literature, where this exact design — multiple algorithms compared across multiple datasets — is common; see Demšar, 2006, in the references).

```
# `apply(..., 1, rank)` ranks the four algorithm scores within each row
# (i.e., within each dataset/block); transposing puts datasets back as rows.
rank_matrix <- t(apply(ml_matrix, 1, rank))
colnames(rank_matrix) <- colnames(ml_matrix)

rank_matrix
```

```
#>           Logistic Regression Random Forest SVM Gradient Boosting
#> Dataset_1                1                3  2                4
#> Dataset_2                1                3  2                4
#> Dataset_3                1                3  2                4
#> Dataset_4                2                3  1                4
```

```
#> Dataset_5          1          3  2          4
#> Dataset_6          1          3  2          4
#> Dataset_7          2          3  1          4
#> Dataset_8          1          3  2          4
#> Dataset_9          1          3  2          4
#> Dataset_10         1          3  2          4
```

```
colMeans(rank_matrix)
```

```
#> Logistic Regression      Random Forest      SVM      Gradient Boosting
#>                1.2                3.0                1.8                4.0
```

4.3 Effect Size: Kendall's W

The chi-squared statistic and p-value indicate whether the algorithms' ranks differ by more than chance, but not how strongly the blocks agree on the ranking. Kendall's coefficient of concordance (W) answers that question. W ranges from 0 (no agreement among blocks on how to rank the treatments — equivalent to random rankings) to 1 (perfect agreement — every block ranks the treatments in exactly the same order).

```
# Kendall's W = chi-squared / (N * (k - 1)), where N = number of blocks
# (datasets) and k = number of treatments (algorithms).
N <- nrow(ml_matrix)
k <- ncol(ml_matrix)

kendalls_w <- unname(friedman_result$statistic) / (N * (k - 1))
kendalls_w
```

```
#> [1] 0.936
```

As a rule of thumb, $W < .3$ is commonly described as weak agreement, $.3-.5$ as moderate, and $> .5$ as strong; values above $.9$ indicate the blocks agree on the ranking almost without exception.

4.4 Post-Hoc Test: Nemenyi Test

A significant Friedman test indicates that the algorithms are not all ranked equally across datasets, but, like the omnibus F-test in ANOVA, it does not say which specific pairs of algorithms differ. The Nemenyi test is the standard post-hoc procedure for the Friedman test,

and is the rank-based counterpart to Tukey's HSD: it controls the family-wise error rate across all pairwise comparisons using the studentized range distribution, but applied to rank sums rather than to means.

```
# The critical difference (CD) for the Nemenyi test, following Demsar (2006):
#   CD = (q_{alpha,k,Inf} / sqrt(2)) * sqrt(k(k+1) / (6N))
# `qtukey()` is R's built-in quantile function for the studentized range
# distribution -- the same distribution TukeyHSD() relies on -- so this
# computation needs no extra package beyond base R.
```

```
alpha <- 0.05
```

```
q_crit <- qtkey(1 - alpha, nmeans = k, df = Inf) / sqrt(2)
```

```
critical_difference <- q_crit * sqrt(k * (k + 1) / (6 * N))
```

```
avg_ranks <- colMeans(rank_matrix)
```

```
pairwise_combinations <- combn(colnames(ml_matrix), 2)
```

```
nemenyi_table <- data.frame(
  Comparison = apply(pairwise_combinations, 2, paste, collapse = " vs. "),
  Rank_Diff = apply(pairwise_combinations, 2, function(pair) {
    abs(avg_ranks[pair[1]] - avg_ranks[pair[2]])
  })
)
```

```
nemenyi_table$z <- nemenyi_table$Rank_Diff / sqrt(k * (k + 1) / (6 * N))
```

```
nemenyi_table$p_adj <- sapply(nemenyi_table$z, function(z) 1 - ptukey(z * sqrt(2), nmeans = k))
```

```
nemenyi_table$Significant <- nemenyi_table$Rank_Diff > critical_difference
```

```
critical_difference
```

```
#> [1] 1.483231
```

```
nemenyi_table
```

```
#>           Comparison Rank_Diff      z      p_adj
#> 1 Logistic Regression vs. Random Forest    1.8 3.117691 9.856376e-03
#> 2           Logistic Regression vs. SVM    0.6 1.039230 7.263486e-01
#> 3 Logistic Regression vs. Gradient Boosting    2.8 4.849742 7.362089e-06
#> 4           Random Forest vs. SVM    1.2 2.078461 1.601259e-01
#> 5 Random Forest vs. Gradient Boosting    1.0 1.732051 3.069497e-01
#> 6           SVM vs. Gradient Boosting    2.2 3.810512 7.992656e-04
```

```
#> Significant
#> 1          TRUE
#> 2          FALSE
#> 3          TRUE
#> 4          FALSE
#> 5          FALSE
#> 6          TRUE
```

A pair of algorithms is declared significantly different at $\alpha = .05$ when the absolute difference in their average ranks exceeds the critical difference computed above.

4.5 Alternative Post-Hoc Test: Pairwise Wilcoxon Signed-Rank Tests

The Nemenyi test is the most commonly taught post-hoc procedure for the Friedman test, but it is known in the literature to be conservative (low statistical power), particularly with a small number of blocks. An alternative is to run a paired Wilcoxon signed-rank test on every pair of algorithms (using the original, unranked block-level scores rather than the ranks) and correct the resulting p-values for multiple comparisons, commonly with the Holm method.

```
pairwise.wilcox.test(
  x = ml_blocked_data$Accuracy,
  g = ml_blocked_data$Algorithm,
  paired = TRUE,
  p.adjust.method = "holm",
  exact = FALSE
)
```

```
#>
#> Pairwise comparisons using Wilcoxon signed rank test with continuity correction
#>
#> data: ml_blocked_data$Accuracy and ml_blocked_data$Algorithm
#>
#>
#>           Logistic Regression Random Forest SVM
#> Random Forest    0.036             -          -
#> SVM              0.036             0.036      -
#> Gradient Boosting 0.036             0.036      0.036
#>
#> P value adjustment method: holm
```

This second method is not a replacement for the Nemenyi test; it is shown as an alternative so that the trade-off is visible directly in the output rather than only described in the abstract:

the Wilcoxon-based approach can flag more pairs as significant than the Nemenyi test does on the same data, which is the practical expression of “more power, less conservative.” Neither test is wrong; they encode different assumptions and different tolerances for false positives, and the choice should be stated and justified rather than left implicit.

5 Diagnostic EDA (Model Diagnostic)

5.1 Test of Linearity — Not Applicable

As in the ANOVA notebook, linearity is a property of the relationship between a continuous predictor and an outcome in a regression-style model. Algorithm and Dataset are both categorical group memberships with no inherent numeric scale, so there is no linear (or non-linear) relationship to test.

5.2 Test of Normality — Not Required

This is the central practical difference between the Friedman test and a repeated-measures ANOVA. The Friedman test operates on ranks, not raw values, and its null distribution is derived from the number of ways the ranks within each block could have occurred by chance — it does not assume the underlying Accuracy scores, or the residuals of any model, are normally distributed. No Shapiro-Wilk test or Q-Q plot is required before interpreting the Friedman test result.

This is worth stating explicitly because it is the most common error students make: carrying over the assumption-checking habit of “test normality first” into a test that was chosen specifically to avoid needing that assumption. The block-level data’s skewness and kurtosis were reported earlier in the Initial EDA purely for descriptive completeness, not as a precondition for the Friedman test.

5.3 Test of Homoscedasticity — Not Required

For the same reason, Levene’s test (or any other homogeneity-of-variance test) is not a precondition for the Friedman test. Homogeneity of variance is a parametric assumption tied to comparing means; a rank-based test comparing rank sums has no equivalent requirement.

5.4 Completeness of the Block Design

What the Friedman test *does* require — and what genuinely deserves checking, unlike the two items above — is that the design is complete and balanced: every block must contribute exactly one score to every treatment, with none missing and none duplicated. This was checked visually in the Initial EDA’s missing data plot; the check below confirms it numerically.

```
# `complete.cases()` flags rows with no missing values; if the block design
# is complete, this should be TRUE for every block (row) and the matrix
# should contain no NA values at all.
all(complete.cases(ml_matrix))
```

```
#> [1] TRUE
```

```
sum(is.na(ml_matrix))
```

```
#> [1] 0
```

```
# dim() confirms the expected 10 blocks x 4 treatments shape
dim(ml_matrix)
```

```
#> [1] 10 4
```

5.5 Check for Tied Ranks Within Blocks

Because the Friedman test ranks the treatment scores separately within each block, ties (two or more equal scores within the same block) reduce the test’s effective resolution. R’s `friedman.test()` automatically applies a tie-correction to the chi-squared statistic when ties are present, but it remains good practice to check how many ties exist, since a large number of tied ranks (most often arising from coarse measurement scales, such as Likert ratings) can noticeably reduce the test’s power.

```
# For each block (row), count how many of its 4 values are NOT unique;
# a value of 0 means no ties in that block.
ties_per_block <- apply(ml_matrix, 1, function(row) length(row) - length(unique(row)))
ties_per_block
```

```
#> Dataset_1 Dataset_2 Dataset_3 Dataset_4 Dataset_5 Dataset_6 Dataset_7
#>      0      0      0      0      0      0      0
#> Dataset_8 Dataset_9 Dataset_10
#>      0      0      0
```

```
sum(ties_per_block)
```

```
#> [1] 0
```

With continuous accuracy scores carried to many decimal places, exact ties are unlikely; this dataset has none. Ties become far more common, and far more consequential, with coarser outcome scales (for example, integer ratings from 1 to 5), where the tie-correction becomes practically important rather than a formality.

5.6 Test of Non-Additivity (Block $\times$ Treatment Interaction)

The Friedman test implicitly assumes that the treatment effect is consistent across blocks — formally, that there is no block-by-treatment interaction (non-additivity). If the relative ordering of algorithms changed substantially from one dataset to another (for example, if Algorithm A were best on some datasets but worst on others), summarizing each algorithm by a single average rank across all ten datasets would obscure that interaction rather than detect it, and the omnibus result could be misleading.

This is exactly what the profile plot produced earlier in the Initial EDA is for: lines that run roughly parallel across the four algorithms, for every dataset, are visual evidence against a meaningful interaction. Re-examine that plot with this question specifically in mind: do any datasets show a markedly different ordering of algorithms compared to the rest? A formal statistical test for non-additivity (Tukey’s one-degree-of-freedom test) exists but is not included here, since with only 10 blocks it has very low power and a visual check is the more honest and more commonly used approach at this sample size.

6 Interpretation of the Results

The presentation of the results and its subsequent interpretation is based on the following notes.

Chi-Squared Statistic—²(df): The Friedman test’s chi-squared statistic measures how far the observed rank sums for each algorithm are from the rank sums that would be expected if every ranking of the four algorithms, within every dataset, were equally likely (i.e., if algorithm made no difference to the ordering).

- A χ^2 value close to 0 indicates the observed rank sums are close to what chance alone would produce.
- A larger χ^2 value indicates the rank sums are more unequal than chance would typically produce, providing stronger evidence against the null hypothesis that all algorithms are interchangeable in rank.

Degrees of Freedom—(df): For the Friedman test, $df = k - 1$, where k is the number of treatments (here, $k = 4$ algorithms, so $df = 3$). Unlike the between-groups df in a one-way ANOVA, this value does not depend on the number of blocks (datasets); the number of blocks is instead conventionally reported alongside the statistic, since df alone does not communicate sample size for this test.

Kendall’s W: Functions here as the Friedman test’s effect size, in the same conceptual role that χ^2 plays for ANOVA: it expresses the *strength* of the pattern (how much the blocks agree on the ranking), separately from whether that pattern is statistically distinguishable from chance (which is what χ^2 and p address).

Academic Reporting (Based on the APA 7th Edition Style, Adapted for a Non-Parametric, Rank-Based Test)

1. The type of statistical test must be stated explicitly as a Friedman test (not simply “ANOVA”), since the two tests rest on different assumptions and are not interchangeable in reporting.
2. Report the chi-squared statistic with its degrees of freedom and the number of blocks: $\chi^2(df, N = \text{blocks}) = \text{value}$, since df alone ($k - 1$) does not convey how many blocks contributed to the test, unlike the df reported for ANOVA.
3. Exact p-values should be reported when possible (e.g., $p = .032$), unless they are less than .001, in which case report as $p < .001$.
4. Report Kendall’s W as the effect size, rather than χ^2 or R^2 , since those are defined in terms of variance explained around a mean — a concept that does not transfer to a rank-based test.
5. When the omnibus test is significant, report the post-hoc procedure used (e.g., the Nemenyi test) and either the critical difference or the adjusted p-value for each pairwise comparison, exactly as Tukey’s HSD results are reported after a significant ANOVA.
6. Report average ranks per treatment alongside, or instead of, raw means; ranks are what the test itself operates on, and reporting only the means can be misleading if the underlying distributions are skewed.
7. Two decimal places are typical for test statistics and effect sizes; p-values may need three or more decimal places. Round to the nearest value; do not truncate.

8. Italicize statistical symbols (W , p) but not Greek letters (χ^2) or sample-size notation (N), consistent with general APA convention.

Further reading: <https://apastyle.apa.org/jars>

6.1 Friedman Test and Post-Hoc Comparisons

6.1.1 Technical Academic Report (Based on APA Guidelines of Reporting Results)

A Friedman test was conducted to compare classification accuracy across four algorithms (Logistic Regression, Random Forest, Support Vector Machine, Gradient Boosting), each evaluated on the same ten datasets (blocks). Unlike the parametric tests reported in the ANOVA notebook, no normality or homogeneity-of-variance assumptions were required prior to this test; the design's completeness (one score per block-treatment cell, with no missing combinations) and the absence of tied ranks within blocks were confirmed in the Diagnostic EDA above.

The effect of Algorithm on the rank ordering of accuracy was statistically significant, $\chi^2(3, N = 10) = 28.08$, $p < .001$, Kendall's $W = .94$, indicating near-complete agreement among the ten datasets on how the four algorithms should be ranked.

Average ranks (1 = lowest accuracy, 4 = highest accuracy, within each dataset) were: Gradient Boosting (M rank = 4.00), Random Forest (M rank = 3.00), Support Vector Machine (M rank = 1.80), and Logistic Regression (M rank = 1.20). This produces a strict ordering — Gradient Boosting first, Random Forest second, SVM third, Logistic Regression last — that held with essentially no exceptions across the ten datasets, consistent with the near-parallel lines observed in the profile plot during the Diagnostic EDA.

Post hoc comparisons using the Nemenyi test (critical difference = 1.48 at $\alpha = .05$) indicated that three of the six pairwise comparisons reached statistical significance:

Comparison	Rank Diff.	z	p (adj.)	Significant
Logistic Regression vs. Random Forest	1.80	3.12	.010	Yes
Logistic Regression vs. SVM	0.60	1.04	.726	No
Logistic Regression vs. Gradient Boosting	2.80	4.85	< .001	Yes

Comparison	Rank Diff.	z	p (adj.)	Significant
Random Forest vs. SVM	1.20	2.08	.160	No
Random Forest vs. Gradient Boosting	1.00	1.73	.307	No
SVM vs. Gradient Boosting	2.20	3.81	< .001	Yes

As a sensitivity check, the same six pairs were also compared using paired Wilcoxon signed-rank tests with Holm correction. That procedure flagged all six pairs as significant ($p = .036$ for each), including the three pairs the Nemenyi test did not distinguish (Logistic Regression vs. SVM, Random Forest vs. SVM, and Random Forest vs. Gradient Boosting). This divergence is a direct, real illustration of the conservatism documented for the Nemenyi test in the literature (Demšar, 2006): both procedures agree on the overall ranking, but they disagree on exactly which adjacent pairs clear the threshold for “statistically distinguishable,” because they make different trade-offs between Type I and Type II error at this sample size. Neither result should be treated as more “correct” without first deciding, in advance, which error rate the analysis is meant to protect against.

6.1.2 Non-Technical Business Report

Four algorithms were compared on accuracy, using the same ten benchmark datasets for every algorithm so that the comparison is fair and matched rather than relying on pooled averages. The result is consistent: Gradient Boosting comes out on top in essentially every one of the ten datasets, Random Forest is reliably second, Support Vector Machine third, and Logistic Regression last. The agreement across datasets on this ranking is very strong — this is not a result driven by one or two unusual datasets.

The one place this analysis is less clear-cut than the earlier ANOVA-based comparisons is in the middle of the pack: whether Random Forest is “really” different from SVM, and whether Random Forest is “really” different from Gradient Boosting, depends on which statistical method is used to make that specific call. One method (the more cautious one) says no, those two pairs are too close to call with confidence; a second, slightly less cautious method says yes, the difference holds up. The gap between Gradient Boosting and Logistic Regression at the extremes, by contrast, is not in question under either method.

6.1.3 Recommendations

1. **Treat the overall ranking (Gradient Boosting > Random Forest > SVM > Logistic Regression) as well-supported.** It is corroborated by two independent lines of evidence in this notebook: the parametric one-way ANOVA in the earlier notebook and the non-parametric Friedman test here, despite the two tests resting on different assumptions about the data.
2. **Decide, before reporting, which post-hoc method’s significance calls will be treated as authoritative** for the Random Forest–SVM and Random Forest–Gradient Boosting comparisons, since the Nemenyi and Wilcoxon-based procedures disagree on these two specific pairs. State the choice and the reason for it (typically, the desired balance between Type I and Type II error) rather than selectively reporting whichever procedure supports a preferred conclusion.
3. **Treat the Random Forest vs. Gradient Boosting comparison as the genuine decision point**, exactly as the ANOVA notebook’s recommendations already noted from the parametric analysis — this notebook’s results reinforce that conclusion rather than overturn it.
4. **Use this design (multiple algorithms, same datasets, Friedman test) as the default comparison method for future algorithm benchmarking**, in preference to treating repeated evaluations on shared datasets as independent observations, since the latter understates how related the observations actually are.
5. **Re-run this analysis with a larger number of datasets if the Random Forest–SVM and Random Forest–Gradient Boosting distinction needs to be resolved with confidence**, since both the Nemenyi test’s conservatism and the Wilcoxon test’s “floor effect” observed above (several pairs sharing an identical adjusted p-value of .036) are symptomatic of the limited statistical power available with only ten blocks.

7 References and Further Reading

American Psychological Association. (2025, February). *Journal Article Reporting Standards (JARS)*. APA Style. Retrieved April 28, 2025, from <https://apastyle.apa.org/jars>

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30. <https://www.jmlr.org/papers/v7/demsar06a.html>

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200), 675–701. <https://doi.org/10.1080/01621459.1937.10503522>

Hodeghatta, U. R., & Nayak, U. (2023). *Practical Business Analytics Using R and Python: Solve Business Problems Using a Data-driven Approach* (2nd ed.). Apress. <https://link.springer.com/book/10.1007/978-1-4842-8754-5>